

Cahier des Charges

User Management & Subscription Service

Plateforme CAPQUANT

Stack Technique : C++17

Framework : Crow / Boost.Beast

Base de données : PostgreSQL

Version 1.0
6 octobre 2025

Document Confidentiel

Table des matières

1	VISION & MISSION STRATÉGIQUE	3
1.1	Contexte	3
1.2	Votre Mission	3
2	PÉRIMÈTRE FONCTIONNEL & ARCHITECTURE	3
2.1	Position dans l'Architecture	3
2.2	Ce que votre service NE FAIT PAS	3
3	STACK TECHNIQUE	3
3.1	Technologies Principales	3
3.2	Librairies Clés	4
4	LIVRABLE 1 : LA BASE DE DONNÉES	4
4.1	Table users	4
4.1.1	Schéma SQL	4
4.1.2	Description des Colonnes	4
4.2	Table subscriptions	5
4.2.1	Schéma SQL	5
4.2.2	Description des Colonnes	6
5	LIVRABLE 2 : L'API INTERNE EN C++	6
5.1	Endpoint 1 : POST /internal/auth/register	6
5.1.1	Description	6
5.1.2	Implémentation	6
5.1.3	Requête	7
5.1.4	Réponse	7
5.2	Endpoint 2 : POST /internal/auth/login	7
5.2.1	Description	7
5.2.2	Implémentation	7
5.2.3	Requête	7
5.2.4	Réponse	8
5.3	Endpoint 3 : POST /internal/auth/validate-token	8
5.3.1	Description	8
5.3.2	Implémentation	8
5.3.3	Requête	8
5.3.4	Réponse	8
5.4	Endpoint 4 : GET /internal/users/{user_id}/permissions	9
5.4.1	Description	9
5.4.2	Implémentation	9
5.4.3	Requête	9
5.4.4	Réponse	9
6	COLLABORATIONS	10
6.1	Avec l'API Gateway	10
6.2	Avec le Service de Brahim (Asset & Orchestration)	10
6.3	Avec le Frontend	10
6.4	Documentation de l'API	10
7	EXIGENCES NON FONCTIONNELLES	10

7.1	Sécurité	10
7.2	Performance	10
7.3	Fiabilité	11
8	LIVRABLES ATTENDUS	11
9	CRITÈRES DE SUCCÈS	11
9.1	Fonctionnels	11
9.2	Techniques	11
9.3	Opérationnels	12
10	PLANNING INDICATIF	12
11	GLOSSAIRE	12

1 VISION & MISSION STRATÉGIQUE

1.1 Contexte

Dans une architecture de microservices, il est essentiel d'avoir un service centralisé, sécurisé et fiable qui gère l'identité des utilisateurs. Ce service doit répondre à deux questions fondamentales pour tous les autres services :

- “**Qui est cette personne ?**”
- “**Qu’a-t-elle le droit de faire ?**”

1.2 Votre Mission

Votre mission est de construire ce **service d'identité**. Vous êtes le gardien des données les plus sensibles de nos utilisateurs (identifiants, mots de passe) et de la logique commerciale de nos abonnements.

Votre service doit être irréprochable en termes de :

- **Sécurité** : Protection des données sensibles
- **Performance** : Faible latence pour tous les endpoints
- **Fiabilité** : Disponibilité maximale

Toute la plateforme dépend de ce service pour fonctionner.

2 PÉRIMÈTRE FONCTIONNEL & ARCHITECTURE

2.1 Position dans l'Architecture

Type

Service API Interne. Il n'est **jamais** exposé publiquement.

Repository

Il doit être dans son **propre repo distinct**, car c'est un service fondamental qui pourra être partagé par d'autres plateformes (TP, site web principal) à l'avenir.

Dépendances

Il n'a presque aucune dépendance, mais tous les autres services dépendent de lui.

Clients Principaux — L'API Gateway (pour l'authentification)

- L'Asset & Orchestration Service (pour la vérification des quotas)

2.2 Ce que votre service NE FAIT PAS

- Il ne gère aucune logique de trading
- Il ne sait rien des stratégies ou des prédictions
- Il ne gère pas les informations de CRM (profil détaillé, paiements, etc.)

3 STACK TECHNIQUE

3.1 Technologies Principales

Langage

C++17 (ou plus récent), pour la performance et la cohérence avec les autres services critiques.

Framework API

Une librairie C++ performante pour les API REST

— Exemples : Crow, Boost.Beast

Base de Données

PostgreSQL, via une librairie C++ robuste

— Exemple : libpqxx

3.2 Bibliothèques Clés**Librairie de Hachage**

Une implémentation C++ de bcrypt

— Exemple : bcrypt.cpp de GitHub

Librairie JWT

jwt-cpp pour la création et la validation des tokens

Librairie JSON

nlohmann-json pour la manipulation des requêtes et des réponses

4 LIVRABLE 1 : LA BASE DE DONNÉES

Important : Vous êtes le propriétaire exclusif des schémas de ces deux tables.

4.1 Table users

Rôle : Stocker les informations d'identité critiques et sécurisées.

4.1.1 Schéma SQL

Listing 1 – Création de la table users

```

1 CREATE TABLE users (
2     user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     email VARCHAR(255) UNIQUE NOT NULL,
4     password_hash TEXT NOT NULL,
5     -- Utiliser TEXT pour ne pas être limité en taille
6     -- par le hash bcrypt
7     role VARCHAR(20) NOT NULL DEFAULT 'user'
8     CHECK (role IN ('user', 'admin', 'support', 'marketing')),
9     is_active BOOLEAN NOT NULL DEFAULT TRUE,
10    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
11    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
12 );
13
14 -- Index pour des recherches rapides par email
15 CREATE INDEX idx_users_email ON users (email);

```

4.1.2 Description des Colonnes**user_id**

Identifiant unique de l'utilisateur (UUID)

email

Adresse email unique de l'utilisateur

password_hash

Hash bcrypt du mot de passe (stocké en TEXT)

role Rôle de l'utilisateur (user, admin, support, marketing)

is_active

Statut actif/inactif du compte

created_at

Date de création du compte

updated_at

Date de dernière mise à jour

4.2 Table subscriptions

Rôle : Gérer l'état de l'abonnement et les quotas de chaque utilisateur.

4.2.1 Schéma SQL

Listing 2 – Création de la table subscriptions

```

1 CREATE TABLE subscriptions (
2   subscription_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   user_id UUID UNIQUE NOT NULL
4   REFERENCES users(user_id) ON DELETE CASCADE,
5   plan_name VARCHAR(50) NOT NULL DEFAULT 'FREE'
6   CHECK (plan_name IN ('FREE', 'PRO', 'ELITE')),
7   status VARCHAR(50) NOT NULL DEFAULT 'active'
8   CHECK (status IN ('active', 'past_due', 'canceled')),
9
10  -- Colonnes de quotas definis par le plan
11  backtests_per_day_limit INT NOT NULL DEFAULT 5,
12  api_requests_per_hour_limit INT,
13  -- NULL pour les plans non-pro
14
15  -- Suivi de l'utilisation
16  backtests_used_today INT NOT NULL DEFAULT 0,
17  quota_reset_at TIMESTAMP WITH TIME ZONE,
18
19  -- Dates de l'abonnement
20  start_date TIMESTAMP WITH TIME ZONE,
21  end_date TIMESTAMP WITH TIME ZONE,
22  provider_subscription_id TEXT,
23  -- ID de l'abonnement chez Stripe, par exemple
24
25  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
26 );
27
28 -- Index pour retrouver rapidement l'abonnement d'un utilisateur
29 CREATE INDEX idx_subscriptions_user_id ON subscriptions (user_id);

```

4.2.2 Description des Colonnes

subscription_id

Identifiant unique de l'abonnement (UUID)

user_id

Référence vers l'utilisateur (clé étrangère unique)

plan_name

Type de plan (FREE, PRO, ELITE)

status

Statut de l'abonnement (active, past_due, canceled)

backtests_per_day_limit

Limite de backtests par jour

api_requests_per_hour_limit

Limite de requêtes API par heure (optionnel)

backtests_used_today

Nombre de backtests utilisés aujourd'hui

quota_reset_at

Date de réinitialisation du quota

start_date

Date de début de l'abonnement

end_date

Date de fin de l'abonnement

provider_subscription_id

ID externe (Stripe, etc.)

updated_at

Date de dernière mise à jour

5 LIVRABLE 2 : L'API INTERNE EN C++

Votre service exposera une API REST interne robuste.

5.1 Endpoint 1 : POST /internal/auth/register

5.1.1 Description

Création d'un nouveau compte utilisateur avec initialisation de l'abonnement par défaut.

5.1.2 Implémentation

1. Reçoit `email` et `password`
2. Utilise la librairie `bcrypt` pour hacher le mot de passe
3. Crée une entrée dans la table `users`
4. Crée une entrée par défaut dans la table `subscriptions`

5.1.3 Requête

Listing 3 – Exemple de requête

```
1 POST /internal/auth/register
2 Content-Type: application/json
3
4 {
5     "email": "user@example.com",
6     "password": "SecurePassword123!"
7 }
```

5.1.4 Réponse

Listing 4 – Exemple de réponse

```
1 {
2     "success": true,
3     "user_id": "123e4567-e89b-12d3-a456-426614174000",
4     "email": "user@example.com",
5     "role": "user"
6 }
```

5.2 Endpoint 2 : POST /internal/auth/login

5.2.1 Description

Authentification d'un utilisateur et génération d'un token JWT.

5.2.2 Implémentation

1. Reçoit `email` et `password`
2. Récupère le `password_hash` depuis la table `users`
3. Compare le mot de passe fourni avec le hash
4. Si correct, utilise `jwt-cpp` pour générer un token JWT contenant `user_id` et `role` dans son payload

5.2.3 Requête

Listing 5 – Exemple de requête

```
1 POST /internal/auth/login
2 Content-Type: application/json
3
4 {
5     "email": "user@example.com",
6     "password": "SecurePassword123!"
7 }
```


5.2.4 Réponse

Listing 6 – Exemple de réponse

```
1 {  
2   "success": true,  
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
4   "user_id": "123e4567-e89b-12d3-a456-426614174000",  
5   "role": "user"  
6 }
```

5.3 Endpoint 3 : POST /internal/auth/validate-token

5.3.1 Description

Endpoint le plus important du service. Validation d'un token JWT et récupération des permissions complètes de l'utilisateur.

5.3.2 Implémentation

1. Reçoit un token JWT
2. Utilise jwt-cpp pour valider la signature et l'expiration
3. Fait une requête SQL pour récupérer les informations de l'utilisateur et de son abonnement
4. Retourne un objet JSON complet avec les permissions et les quotas actuels

Exigence critique : Cet endpoint doit être extrêmement optimisé pour une faible latence.

5.3.3 Requête

Listing 7 – Exemple de requête

```
1 POST /internal/auth/validate-token  
2 Content-Type: application/json  
3  
4 {  
5   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
6 }
```

5.3.4 Réponse

Listing 8 – Exemple de réponse

```
1 {  
2   "valid": true,  
3   "user_id": "123e4567-e89b-12d3-a456-426614174000",  
4   "email": "user@example.com",  
5   "role": "user",  
6   "is_active": true,  
7   "subscription": {  
8     "plan_name": "PRO",  
9     "status": "active",  
10    "backtests_per_day_limit": 50,  
11    "backtests_used_today": 12,  
12  }
```

```
12     "api_requests_per_hour_limit": 1000,  
13     "quota_reset_at": "2025-10-07T00:00:00Z"  
14 }  
15 }
```

5.4 Endpoint 4 : GET /internal/users/{user_id}/permissions

5.4.1 Description

Récupération des permissions et quotas d'un utilisateur spécifique.

5.4.2 Implémentation

1. Reçoit un `user_id`
2. Fait une requête JOIN sur les tables `users` et `subscriptions`
3. Retourne un résumé des droits et des quotas

5.4.3 Requête

Listing 9 – Exemple de requête

```
1 GET /internal/users/123e4567-e89b-12d3-a456-426614174000/permissions
```

5.4.4 Réponse

Listing 10 – Exemple de réponse

```
1 {  
2   "user_id": "123e4567-e89b-12d3-a456-426614174000",  
3   "role": "user",  
4   "is_active": true,  
5   "permissions": {  
6     "can_backtest": true,  
7     "can_use_api": true,  
8     "is_admin": false  
9   },  
10  "quotas": {  
11    "backtests_per_day_limit": 50,  
12    "backtests_used_today": 12,  
13    "backtests_remaining_today": 38,  
14    "api_requests_per_hour_limit": 1000  
15  },  
16  "plan_name": "PRO",  
17  "plan_status": "active"  
18 }
```

6 COLLABORATIONS

6.1 Avec l'API Gateway

Vous êtes son fournisseur d'identité. Il est le seul client de vos endpoints :

- `/register`
- `/login`
- `/validate-token`

6.2 Avec le Service de Brahim (Asset & Orchestration)

Il est le client de votre endpoint `/permissions` pour vérifier les quotas des utilisateurs avant de lancer des tâches lourdes.

6.3 Avec le Frontend

Aucun lien direct. Toutes les communications passent par l'API Gateway.

6.4 Documentation de l'API

Vous devez fournir une spécification OpenAPI (Swagger) interne et la partager avec :

- Les développeurs de l'API Gateway
- Les développeurs du service de Brahim

7 EXIGENCES NON FONCTIONNELLES

7.1 Sécurité

- **Hachage des mots de passe** : Utilisation obligatoire de bcrypt avec un facteur de coût approprié (minimum 10)
- **Tokens JWT** : Signature sécurisée, expiration courte (recommandé : 1 heure)
- **Validation stricte** : Validation de tous les inputs (email, password, tokens)
- **Protection SQL Injection** : Utilisation de requêtes préparées exclusivement
- **Logs sécurisés** : Aucun mot de passe ou token ne doit apparaître dans les logs

7.2 Performance

- **Latence** : L'endpoint `/validate-token` doit répondre en moins de 50ms (P95)
- **Débit** : Le service doit supporter au minimum 1000 requêtes par seconde
- **Optimisations** :
 - Index sur les colonnes fréquemment utilisées
 - Connection pooling pour PostgreSQL
 - Cache en mémoire pour les tokens récemment validés (optionnel)

7.3 Fiabilité

- **Disponibilité** : 99.9% minimum
- **Gestion des erreurs** : Codes HTTP appropriés et messages d'erreur clairs
- **Monitoring** : Logs structurés et métriques exposées
- **Tests** : Couverture de tests unitaires et d'intégration supérieure à 80%

8 LIVRABLES ATTENDUS

1. **Code Source C++**
 - Architecture propre et modulaire
 - Code documenté (commentaires Doxygen)
 - Respect des standards C++17
2. **Scripts SQL**
 - Création des tables
 - Création des index
 - Migrations éventuelles
3. **Documentation**
 - Spécification OpenAPI/Swagger
 - Guide de déploiement
 - Guide d'utilisation pour les développeurs
4. **Tests**
 - Tests unitaires
 - Tests d'intégration
 - Tests de charge
5. **Configuration**
 - Fichier de configuration (format : JSON ou YAML)
 - Variables d'environnement documentées
 - Dockerfile pour le déploiement

9 CRITÈRES DE SUCCÈS

9.1 Fonctionnels

- Tous les endpoints fonctionnent correctement
- L'authentification est sécurisée et fiable
- Les quotas sont correctement suivis et appliqués
- Les permissions sont correctement vérifiées

9.2 Techniques

- Latence P95 < 50ms pour `/validate-token`
- Débit > 1000 requêtes/seconde
- Couverture de tests > 80%
- Aucune faille de sécurité identifiée

9.3 Opérationnels

- Documentation complète et claire
- Déploiement automatisé
- Monitoring et alerting en place
- Logs structurés et exploitables

10 PLANNING INDICATIF

Semaine 1-2

Mise en place de l'infrastructure et des schémas de base de données

Semaine 3-4

Développement des endpoints d'authentification (`/register`, `/login`)

Semaine 5-6

Développement des endpoints de validation (`/validate-token`, `/permissions`)

Semaine 7

Tests unitaires et d'intégration

Semaine 8

Tests de charge et optimisations

Semaine 9

Documentation et préparation au déploiement

Semaine 10

Déploiement et monitoring

11 GLOSSAIRE

API Gateway

Service qui expose les API publiques et route les requêtes vers les microservices appropriés

bcrypt

Algorithme de hachage de mot de passe adaptatif et sécurisé

JWT (JSON Web Token)

Standard ouvert pour créer des tokens d'accès

Microservice

Architecture où l'application est composée de services indépendants et déployables séparément

PostgreSQL

Système de gestion de base de données relationnelle open source

UUID

Identifiant unique universel (Universally Unique Identifier)

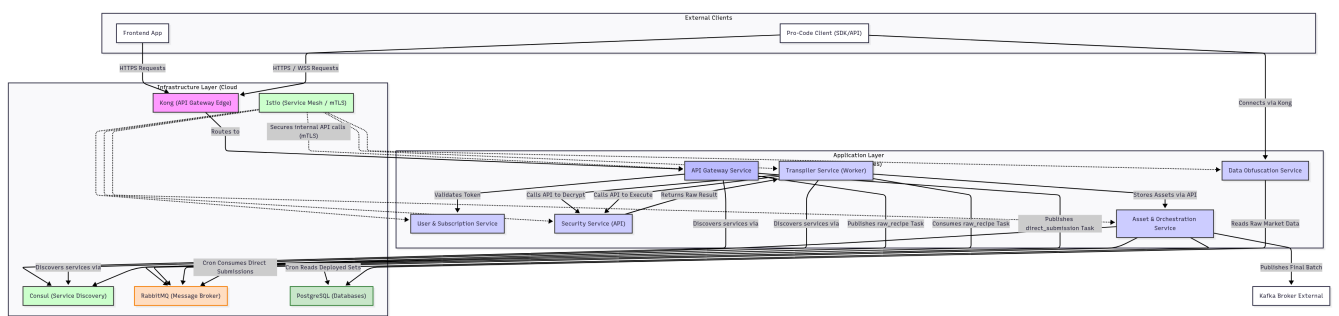


FIGURE 1 – Architecture CQ